are provided on Seqs in the same manner as `createDataFrame(...)` and `toDF()`. For converting from RDD to Dataset you can first convert from RDD to DataFrame and then convert it to a Dataset.

For loading data into a Dataset, unless a special API is provided by your data source, you can first load your data into a DataFrame and then convert it to a Dataset. Since the conversion to the Dataset simply adds information you do not have the problem of eagerly evaluating, and future filters and similar operations can still be pushed down to the data store.

Example 3-44. Create a Dataset from a DataFrame

Converting from a Dataset back to an RDD or DataFrame can be done in similar ways as when converting DataFrames. The `toDF` simply copies the logical plan used in the Dataset into a DataFrame - so you don't need to do any schema inference or conversion as you do when converting from RDDs. Converting a Dataset of type `T` to an RDD of type `T` can be done by calling `.rdd`, which unlike calling `toDF`, does involve converting the data from the internal SQL format to the regular types.

Example 3-45. Convert Dataset to DataFrame & RDD

```
/**
 * Illustrate converting a Dataset to an RDD
 */
def toRDD(ds: Dataset[RawPanda]): RDD[RawPanda] = {
  ds.rdd
}

/**
 * Illustrate converting a Dataset to a DataFrame
 */
def toDF(ds: Dataset[RawPanda]): DataFrame = {
  ds.toDF()
}
```

Compile Time Strong Typing

One of the reasons to use Datasets over traditional DataFrames is their compile time strong typing. DataFrames have runtime schema information but lack compile time information about the schema. This strong typing is especially useful when making libraries, because you can more clearly specify the requirements of your inputs and your return types.

Easier Functional (RDD "like") Transformations

One of the key advantages of the Dataset API is easier integration with custom Scala and Java code. Datasets expose `filter`, `map`, `mapPartitions`, and `flatMap` with similar function signatures as RDDs, with the notable requirement that our return ElementType also be understandable by Spark SQL (such as tuple or case class of types discussed in "Basics of Schemas" on page 41).

Example 3-46. Create a Dataset from a DataFrame

```
def fromDF(df: DataFrame): Dataset[RawPanda] = {
  df.as[RawPanda]
}
```

Beyond functional transformations, such as `map` and `filter`, you can also intermix relational and grouped operations.

Relational Transformations

Datasets introduce a typed version of `select` for relational style transformations. When specifying an expression for this you need to include the type information. You can add this information by calling `as[ReturnType]` on the expression/column.

Example 3-47. Simple relational select on Dataset

```
def squishyPandas(ds: Dataset[RawPanda]): Dataset[(Long, Boolean)] = {
  ds.select($"id".as[Long], ($"attributes"(0) > 0.5).as[Boolean])
}
```

Some operations, such as select, have both typed and untyped implementations. If you supply a `Column` rather than a `TypedColumn` you will get a DataFrame back instead of a Dataset.

Multi-Dataset Relational Transformations

In addition to single Dataset transformations, there are also transformations for working with multiple Datasets. The standard set operations, namely `intersect`, `union`, and `subtract`, are all available with the same standard caveats as discussed in Table 3-9. Joining Datasets is also supported, but to make the type information easier to work with, the return structure is a bit different than traditional SQL joins.

Grouped Operations on Datasets

Similar to grouped operations on DataFrames, `groupBy` on Datasets return a `GroupedDataset`, on which you can specify your aggregate functions, along with a functional `mapGroups` API. As with the expression in "Relational Transformations" on page 71, you need to use typed expressions so the result can also be a Dataset. Taking our previous example of computing the maximum panda size by zip in Example 3-18, you would rewrite it to be as shown in Example 3-48.

The convenience functions found on `GroupedData` (e.g. `min`, `max`, etc.) are missing, so all of our aggregate expressions need to be specified through `agg`.

Example 3-48. Compute the max panda size per zip code typed

```
def maxPandaSizePerZip(ds: Dataset[RawPanda]): Dataset[(String, Double)] = {
  ds.groupBy($"zip").keyAs[String].agg(max("attributes(2)").as[Double])
}
```

Beyond applying typed SQL expressions to aggregated columns, you can also easily use arbitrary Scala code with `mapGroups` on grouped data as shown in Example 3-49. This can save us from having to write custom user defined aggregate functions (UDAFs) (discussed in "Extending with User Defined Functions & Aggregate Functions (UDFs, UDAFs)" on page 72). While custom UDAFs can be painful to write, they may be able to give better performance than `mapGroups` and can also be used on `DataFrames`.

Example 3-49. Compute the max panda size per zip code using map groups

```
def maxPandaSizePerZipScala(ds: Dataset[RawPanda]): Dataset[(String, Double)] = {
  ds.groupBy($"zip").keyAs[String].mapGroups{ case (g, iter) =>
    (g, iter.map(_.attributes(2)).reduceLeft(Math.max(_, _)))
  }
}
```

Extending with User Defined Functions & Aggregate Functions (UDFs, UDAFs)

User defined functions and user defined aggregate functions provide you with ways to extend the DataFrame & SQL APIs with your own custom code while keeping the Catalyst optimizer. The Dataset API is another performant option for much of what you can do with UDFs and UDAFs. This is quite useful for performance, since otherwise you would need to convert the data to an RDD (and potentially back again) to

perform arbitrary functions, which is quite expensive. UDFs and UDAFs can also be accessed from inside of regular SQL expressions, making them accessible to analysts or others more comfortable with SQL.

When using UDFs or UDAFs written in non-JVM languages, it is important to note that you lose much of the performance benefit, as the data must still be transferred.

If most of your work is in Python but you want to access some UDFs without the performance penalty, you can write your UDFs in Scala and register them for use in Python (as done in Sparkling Pandas).[5]

Writing non-aggregate UDF for Spark SQL is incredibly simple: you simply write a regular function and register it using `sqlContext.udf().register`. If you are registering a Java or Python UDF you also need to specify our return type.

Example 3-50. String length UDF

```
def setupUDFs(sqlCtx: SQLContext) = {
  sqlCtx.udf.register("strLen", (s: String) => s.length())
}
```

Aggregate functions (or UDAFs) are somewhat trickier to write. Instead of writing a regular Scala function, you extend the `UserDefinedAggregateFunction` and implement a number of different functions, similar to the functions one might write for `aggregateByKey` on an RDD, except working with different data structures. While they can be complex to write, UDAFs can be quite performant compared with options like `mapGroups` on `Datasets` or even simply written `aggregateByKey` on `RDDs`. You can then either use the UDAF directly on columns or add it to the function registry as you did for the non-aggregate UDF.

Example 3-51. UDAF for computing the average

```
def setupUDAFs(sqlCtx: SQLContext) = {
  class Avg extends UserDefinedAggregateFunction {
    // Input type
    def inputSchema: org.apache.spark.sql.types.StructType =
      StructType(StructField("value", DoubleType) :: Nil)
```

5 As of this writing, the Sparkling Pandas project development is on-hold but early releases still contains some interesting examples of using JVM code from Python.

```
    def bufferSchema: StructType = StructType(
      StructField("count", LongType) ::
      StructField("sum", DoubleType) :: Nil
    )

    // Return type
    def dataType: DataType = DoubleType

    def deterministic: Boolean = true

    def initialize(buffer: MutableAggregationBuffer): Unit = {
      buffer(0) = 0L
      buffer(1) = 0.0
    }

    def update(buffer: MutableAggregationBuffer,input: Row): Unit = {
      buffer(0) = buffer.getAs[Long](0) + 1
      buffer(1) = buffer.getAs[Double](1) + input.getAs[Double](0)
    }

    def merge(buffer1: MutableAggregationBuffer, buffer2: Row): Unit = {
      buffer1(0) = buffer1.getAs[Long](0) + buffer2.getAs[Long](0)
      buffer1(1) = buffer1.getAs[Double](1) + buffer2.getAs[Double](1)
    }

    def evaluate(buffer: Row): Any = {
      math.pow(buffer.getDouble(1), 1.toDouble / buffer.getLong(0))
    }
  }
  // Optionally register
  val avg = new Avg
  sqlCtx.udf.register("ourAvg", avg)
}
```

This is a little more complicated than our regular UDF, so let's take a look at what the different parts do. You start by specifying what the input type is, then you specify the schema of the buffer you will use for storing the in-progress work. These schemas are specified in the same way as DataFrame and Dataset schemas, as discussed in "Basics of Schemas" on page 41.

From there the rest of the functions are implementing the same functions you use when writing `aggregateByKey` on an RDD, but instead of taking arbitrary Scala objects you work with `Row` and `MutableAggregationBuffer`. The final `evaluate` function takes the `Row` representing the aggregation data and returns the final result.

UDFs, UDAFs, and Datasets all provide ways to intermix arbitrary code with Spark SQL.

Query Optimizer

Catalyst is the Spark SQL query optimizer, which is used to take the query plan and transform it into an execution plan that Spark can run. Much as our transformations on RDDs build up a DAG, as you apply relational and functional transformations on DataFrames/Datasets, Spark SQL builds up a tree representing our query plan, called a logical plan. Spark is able to apply a number of optimizations on the logical plan and can also choose between multiple physical plans for the same logical plan using a cost-based model.

Logical and Physical Plans

The logical plan you construct through transformations on DataFrames/Datasets (or SQL queries) starts out as an unresolved logical plan. Much like a compiler, the Spark optimizer is multi-phased and before any optimizations can be performed, it needs to resolve the references and types of the expressions. This resolved plan is referred to as the logical plan, and Spark applies a number of simplifications directly on the logical plan, producing an optimized logical plan. These simplifications can be written using pattern matching on the tree, such as the rule for simplifying additions between two literals. The optimizer is not limited to pattern matching, and rules can also include arbitrary Scala code.

Once the logical plan has been optimized, Spark will produce a physical plan. The physical plan stage has both rule-based and cost-based optimizations to produce the optimal physical plan. One of the most important optimizations at this stage is predicate pushdown to the data source level.

Code Generation

As a final step, Spark may also apply code generation for the components. Code generation is done using Janino to compile Java code. Earlier versions used Scala's Quasi Quotes [5], but the overhead was too high to enable code generation for small datasets. In some TPCDS queries, code generation can result in >10x improvement in performance.

In some early versions of Spark for complex queries, code generation can cause failures. If you are on an old version of Spark and run into an unexpected failure it can be worth disabling codegen by setting `spark.sql.codegen` or `spark.sql.tungsten.enabled` to false (depending on version).

5 Scala Quasi Quotes are part of Scala's macro system.

JDBC/ODBC Server

Spark SQL provides a JDBC server to allow external tools, such as business intelligence, to work with data accessible in Spark and to share resources. Spark SQL's JDBC server requires that Spark be built with Hive support.

Since the server tends to be long lived and runs on a single context, it can also be a good way to share cached tables between multiple users.

Spark SQL's JDBC server is based on the HiveServer2 from Hive, and most corresponding connectors designed for HiveServer2 can be used directly with Spark SQL's JDBC server. Simba also offers specific drivers for Spark SQL.

The server can either be started from the command line or started using an existing HiveContext. The command line start and stop commands are `./sbin/start-thriftserver.sh` and `./sbin/stop-thriftserver.sh`. When starting from the command line, you can configure the different Spark SQL properties by specifying `--hiveconf property=value` on the command line. Many of the rest of the command line parameters match that of `spark-submit`. The default host and port is `localhost:10000` and can be configured with `hive.server2.thrift.port` and `hive.server2.thrift.bind.host`.

When starting the JDBC server using an existing `HiveContext`, you can simply update the config properties on the context instead of specifying command line parameters.

Example 3-52. Start JDBC server on a different port

```
./sbin/start-thriftserver.sh --hiveconf hive.server2.thrift.port=9090
```

Example 3-53. Start JDBC server on a different port in Scala

```
sqlContext.setConf("hive.server2.thrift.port", "9090")
HiveThriftServer2.startWithContext(sqlContext)
```

When starting the JDBC server on an existing `HiveContext` make sure to shutdown the JDBC server when exiting.

Conclusion

The considerations for using DataFrames/Datasets over RDDs are complex and changing with the rapid development of Spark SQL. One of the cases where Spark SQL can be difficult to use is when the number of partitions needed for different parts of our pipeline changes, or if you otherwise wish to control the partitioner. While RDDs lack the Catalyst optimizer and relational style queries, they are able to work with a wider variety of data types and provide more direct control over certain types of operations. DataFrames and Datasets also only work with a restricted subset of data types - but when our data is in one of these supported classes the performance improvements of using the Catalyst optimizer provide a compelling case for accepting those restrictions.

DataFrames can be used when you have primarily relational transformations, which can be extended with UDFS when necessary. Compared to RDDs, DataFrames benefit from the efficient storage format of Spark SQL, the Catalyst optimizer, and the ability to perform certain operations directly on the serialized data. One drawback to working with DataFrames is that they are not strongly typed at compile time, which can lead to errors with incorrect column access and other simple mistakes.

Datasets can be used when you want a mix of functional and relational transformations while benefiting from the optimizations for DataFrames and are, therefore, a great alternative to RDDs in many cases. As with RDDs, Datasets are parameterized on the type of data contained in them, which allows for strong compile time type checking but requires that you know our data type at compile time (although `Row` or other generic type can be used). The additional type safety of Datasets can be beneficial even for applications that do not need the specific functionality of DataFrames. One potential drawback is that the Dataset API is continuing to evolve, so updating to future versions of Spark may require code changes.

Pure RDDs work well for data that does not fit into the Catalyst optimizer. RDDs have an extensive and stable functional API, and upgrades to newer versions of Spark are unlikely to require substantial code changes. RDDs also make it easy to control partitioning, which can be very useful for many distributed algorithms. Some types of operations, such as multi-column aggregates, complex joins, and windowed operations, can be daunting to express with the RDD API. RDDs can work with any Java or Kryo serializable data, although the serialization is more often more expensive and less space efficient than the equivalent in DataFrames/Datasets.

Now that you have a good understanding of Spark SQL, it's time to continue on to joins, for both RDDs and Spark SQL.